talk.tex

$ whoami
Andi Skrgat, Memgraph

$ cat title.txt

Inside the Memgraph Kubernetes Operator

How operators work, and how memgraph/kubernetes-operator is built

$

```
$ ls talk/
$ cd Kubernetes and operators/ 
```

$ Why an operator, not just a Helm chart?

Helm chart

the operator's little sibling

- > Renders templates at install / upgrade
- > Applies the YAML, then walks away
- > Day-2 work stays with the user: flags, restarts, upgrades



Operator

an advanced Memgraph user, on duty 24/7

- > Knows how to change a flag and restart safely
- > Rolls pods one by one: no-downtime upgrades
- > Watches continuously and fixes drift

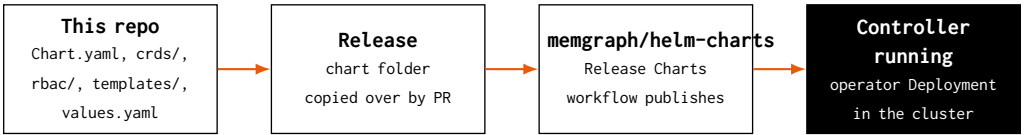
Helm describes what to deploy. The operator knows how to run it.

$ What is an operator, really?

Nothing more than a Go binary that watches the Kubernetes API and acts on it.



$ charts/: install the operator with Helm




```
helm install memgraph-operator memgraph/memgraph-operator
```

One command for the user. Helm installs the operator; the operator then runs Memgraph.

$ Kubernetes is built to be extended

Built-in resources

what we query and use every day

Pod · Service · Deployment
StatefulSet · ConfigMap · Secret
PersistentVolumeClaim · . . .

```
$ kubectl get pods
```

Our own resources

added through a CustomResourceDefinition

MemgraphCluster (memgraph.com)
same REST API, RBAC, kubectl, watch
stored in etcd by the API server

```
$ kubectl get mgc
```

A CRD alone is just data.
Operator = custom resource + a controller that acts on it. That controller is what this talk is about.

$ A CRD is itself a Kubernetes resource

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: memgraphclusters.memgraph.com
spec:
  group: memgraph.com
  names:
    kind: MemgraphCluster
    listKind: MemgraphClusterList
    plural: memgraphclusters
    singular: memgraphcluster
    shortNames: [mgc]
  scope: Namespaced
  versions:
  - name: v1alpha1
    served: true
    storage: true
    schema:
      openAPIV3Schema: { ... }
    subresources:
      status: {}
```

apiVersion	apiextensions.k8s.io/v1
kind	CustomResourceDefinition
metadata	name must be <plural>.<group>
spec	group, names, scope, versions + schema

We don't write it by hand. controller-gen generates it from the Go types and markers: 3,270 lines in config/crd/bases, copied into the Helm chart's crds/. Apply it, and the API server starts serving /apis/memgraph.com/v1alpha1/ . . . /memgraphclusters.

$ The OpenAPI schema guards the door

Without a schema

The API server would accept a MemgraphCluster with any fields (typos, coordinators: "lots") and store it in etcd. The controller finds out later, or never.

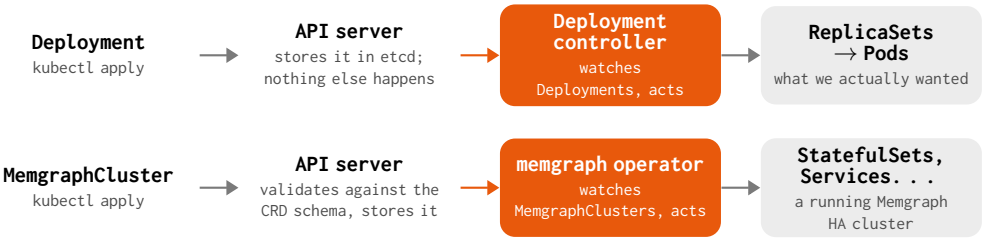
With openAPIV3Schema on the CRD

- > Types, minimum, pattern, enum, CEL rules: checked on every create and update
- > Invalid objects get 422 and never reach etcd
- > Defaults filled in; unknown fields pruned
- > Generated from markers: +kubebuilder:validation:Minimum=3

```
# spec.versions[].schema.openAPIV3Schema
coordinators:
  type: integer
  format: int32
  default: 3
  minimum: 3
  x-kubernetes-validations:
  - rule: self % 2 == 1
    message: coordinators must be an odd
      number so the Raft quorum cannot split
```

```
$ kubectl apply -f cluster.yaml  # coordinators: 4
The MemgraphCluster "my-cluster" is invalid:
spec.coordinators: Invalid value: "integer":
coordinators must be an odd number so the
Raft quorum cannot split
```

$ A resource does nothing without its controller



Without the controller, all we have is a record in etcd. The CRD gives us a validated record of intent; the controller is what turns it into reality. That's the operator.

$ A node dies: controllers all the way down

1
Node dies
the pod on it is deleted: a watch event

2
Owner link
enqueues the pod's parent (its StatefulSet)

3
Parent reconciles
one pod missing → creates a replacement, not yet scheduled

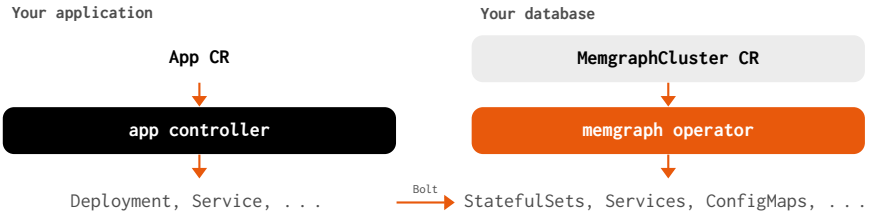
4
Scheduler
a controller: sees an unscheduled pod, binds it to a node

5
kubelet
a controller: sees a pod bound to its node, starts it

No orchestrator
Each step is a separate controller watching its own objects and reacting to the previous one's output. The owner link is what connects step 1 to step 3.

Where we come in
The StatefulSet controller replaces the pod, not us. We see the StatefulSet (owned) and the pod (by label) change, and converge the Memgraph side once it's back. StatefulSet pods are recreated only after the old one is really gone.

$ The pattern: one CR per thing you run




```
kind: MemgraphCluster
spec:
  coordinators: 3
  dataInstances: 2
  image: {tag: 3.13.0-relwithdebinfo}
  secrets: {name: memgraph-secrets}
```

We know Memgraph better than our users. So we decide what the CRD exposes and derive the rest: flags, ports, roll order, registration, TLS wiring. Users think about their application, not about modelling a database.

$ The libraries underneath



mgr.GetClient(): reads come from the in-memory cache, kept in sync by client-go shared informers under the covers. Writes go to the API server.

$ ls talk/

$ cd Under the hood/ 

$ Repository map

api/

CRD Go types and API versions

charts/memgraph-operator/

Install the operator as a Helm chart

cmd/

main.go: the entry point

internal/

controller, planner, rollout, resources. . .

config/

Kustomize: CRD, RBAC, manager manifests

test/e2e/

End-to-end tests on kind

Skeleton scaffolded with Kubebuilder (go/v4 layout): project layout, cmd/main.go, API type stubs, config/ manifests and the Makefile. The PROJECT file records it.

$ api/: one group, evolving versions

```
apiVersion: memgraph.com/v1alpha1
kind: MemgraphCluster
```

Group memgraph.com stays the same, now and in the future.
Version moves forward as the API stabilises:



$ What the version promises

TODAY

v1alpha1

For breaking changes. We stay here as long as we still remove or rename fields, restructure, change a field's type, or change what a default means.

NEXT

v1alpha2. . . / v1beta1

Schema settling. Breaking changes become rare and come with a migration path.

STABLE

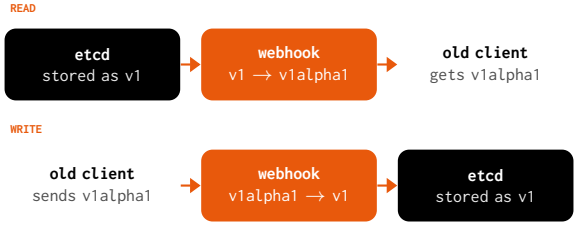
v1

A promise. v1 → v2 should happen rarely, if ever; old versions keep being served long after deprecation.

Why it matters: users' YAML, GitOps repos and client code all pin an apiVersion. Once we say v1, they expect it to keep working, so we get our breaking changes done while we're still alpha.

$ Many versions, one object in etcd

```
# CRD spec
versions:
- name: v1alpha1
  served: true
  storage: false
- name: v1
  served: true
  storage: true # exactly one
conversion:
  strategy: Webhook
```



Hub and spoke
Kubebuilder: v1 is the Hub; v1alpha1 implements ConvertTo / ConvertFrom; controller-runtime serves /convert.

Old objects stay old
The storage version is a choice. Objects written earlier stay in their old version until rewritten (status.storedVersions); migrate before dropping one.

Now in the read path
The webhook needs its own TLS certificate, and if it's down, MemgraphClusters can't be read or written.

$ Round trips without losing fields

The problem
v1 adds a field v1alpha1 doesn't have. An old client reads (converted down: field dropped), edits, writes back (converted up): the field is gone.

Cluster API's answer
On the way down, stash the whole hub object as JSON in an annotation:
`cluster.x-k8s.io/conversion-data`
On the way up, restore the fields the old version can't hold. Helpers: MarshalData / UnmarshalData.

```
// v1alpha1 is a spoke; v1 is the hub
func (dst *MemgraphCluster) ConvertFrom(h conversion.Hub) error {
    src := h.(*v1.MemgraphCluster)
    // ...copy the common fields down...
    return utilconversion.MarshalData(src, dst)
}

func (src *MemgraphCluster) ConvertTo(h conversion.Hub) error {
    dst := h.(*v1.MemgraphCluster)
    // ...copy the common fields up...
    restored := &v1.MemgraphCluster{}
    ok, err := utilconversion.UnmarshalData(src, restored)
    if err != nil || !ok {
        return err
    }
    dst.Spec.NewField = restored.Spec.NewField
    return nil
}
```

$ When do you need the annotation?

Only when all four hold:

1 Two versions served at once
e.g. v2 is out, v1 still in use

2 Different schemas
so there is a conversion webhook

3 Data with no counterpart
one version holds a field the other can't express, and no mapping exists

4 The other version is frozen
you won't add the field there: v1 is a stability promise

Annotations are meant for third parties; this borrows them, but it's a common pattern.

v2 adds spec.backup, v1 is frozen
Stored as v2.

> Read as v1 (v2 → v1): move backup into the annotation.

> Write from v1 (v1 → v2): restore backup from the annotation.

v2 drops spec.legacyMode, v1 still has it
Stored as v2, which has no such field.

> Write from v1 (v1 → v2): move legacyMode into the annotation.

> Read as v1 (v2 → v1): restore it from the annotation.

$ Usually, a conversion function is enough

Field renamed	map old name ↔ new name in the conversion function
Restructured (nested two deep ↔ one deep)	map the paths in the conversion function
Type changed, but derivable	convert the value in the conversion function
New field with a default	absent in the old version means the default: nothing to store
New optional field, old version still evolving	add the field to the old version too
Only one version served	no conversion at all

The annotation is only for data that has nowhere to go.

$ api/v1alpha1/: three files

1 **groupversion_info.go**

Declares GroupVersion {memgraph.com, v1alpha1} and AddToScheme, so main.go can register our types.

2 **memgraphcluster_types.go**

Every API type we use: MemgraphCluster, its Spec and Status and all nested specs (∼2,200 lines). Kubebuilder markers generate the CRD (short name mgc).

3 **zz_generated.deepcopy.go**

Auto-generated Go scaffolding (DeepCopy methods) by controller-gen. Never edited by hand.

$ Kubebuilder markers: what controller-gen reads

```
// +kubebuilder:object:generate=true
// +groupName=memgraph.com
```

object:generate=true make generate writes the DeepCopy methods into zz_generated.deepcopy.go

groupName sets the API group: memgraph.com/v1alpha1

Package level · groupversion_info.go:18

```
// +kubebuilder:object:root=true
// +kubebuilder:subresource:status
// +kubebuilder:resource:shortName=mgc
// +kubebuilder:printcolumn:name="Coordinators",type=integer,JSONPath=`.spec.coordinators`
// +kubebuilder:printcolumn:name="Main",type=string,JSONPath=`.status.main`
// +kubebuilder:printcolumn:name="Ready",type=string,JSONPath=`.status.conditions[?(@.type=="Ready")].status`
// +kubebuilder:printcolumn:name="Registered-Coordinators",type=integer,JSONPath=`.status.coordinators`,priority=1
```

Type level · memgraphcluster_types.go:2219 (more printcolumns in the file)

object:root

Top-level object, so it implements runtime.Object.

subresource:status

Turns on /status: written with r.Status().Patch, never bumps generation.

shortName=mgc

Lets you type kubectl get mgc.

printcolumn

Columns kubectl get shows; priority=1 only with -o wide.

$ Kubernetes API conventions

They ease client development and make configuration mechanisms work consistently across a diverse set of use cases.

RESTful

JSON over standard HTTP verbs:

GET POST PUT PATCH DELETE

plus endpoints for non-standard verbs.

Every JSON in or out has a schema, identified by kind and apiVersion.

```
apiVersion: memgraph.com/v1alpha1
kind: MemgraphCluster
```

Kind The name of a particular object schema.

`Pod, MemgraphCluster`

Resource An entity sent or retrieved as JSON over HTTP: a collection (list of one type) or an element (one). A resource serves one kind; a kind can be served by many.

`Pod ← pods (users), pods/status (automation)`

API group Resources exposed together, versioned as group/version in apiVersion, so groups evolve independently. Each version has one or more resources.

`apps/v1, memgraph.com/v1alpha1`

$ Three categories of kinds

1 Objects

A persistent entity. Creating one is a record of intent: the system works to ensure it exists.

```
kind: MemgraphCluster
```

Ours: MemgraphCluster, the record of intent for one Memgraph HA cluster.

2 Lists

A collection of resources, usually of one kind (can be mixed).

```
kind: MemgraphClusterList
```

Ours: MemgraphClusterList: at informer start, one LIST fetches every cluster; each is stored in the cache as its own object.

3 Simple kinds

Specific actions on objects and non-persistent entities. Many are subresources, rooted at a resource's path.

```
/status, /scale, Status
```

Ours: /memgraphclusters/status, like /pods/status

/pods vs /pods/status. Writes to /pods change metadata and spec; status is changed only through /status, by the processes that observe the world, for users to see. Same split for our status (subresource:status).

$ Every object has metadata

```
metadata:
  namespace: memgraph          MUST
  name: my-cluster             MUST
  uid: 5f1c9e. . .             MUST
  resourceVersion: "48213"     SHOULD
  generation: 3                SHOULD
  creationTimestamp: . . .     SHOULD
  deletionTimestamp: . . .     SHOULD
  labels: {. . .}              SHOULD
  annotations: {. . .}         SHOULD
```

MUST: namespace, name, uid

uid tells apart objects with the same name that were deleted and recreated. Like Memgraph: a name and a UUID, and the UUID is the source of truth. (Our expectations record the StatefulSet's UID for this reason.)

SHOULD

resourceVersion opaque string: the internal version of the object

generation sequence number of the desired state (spec)

creation/deletionTimestamp created / asked to be deleted

labels organizing and selecting, by end users

annotations for third-party automation and tooling

$ spec and status

spec: the desired state

- A complete description: user config, defaults expanded by the system, fields set later by other components
- Persisted with the object; delete the spec and the object is purged
- Declarative names: replicas: 3, never addReplicas: 2

status: the observed state

- Summarizes the current state of the object in the system
- Usually persisted by automated processes (us, via /status); may be generated on the fly
- The most recent observations, as close as possible to the newest spec

Level-based, not edge-based



Update 2 → 5 → 3. Nothing guarantees anyone ever sees 5; only the end state, 3 pods, matters. Missing intermediate changes is harmless.

$ Subresources: one object, two halves

subresources: status: {} from +kubebuilder:subresource:status: two endpoints

.../memgraphclusters/my-cluster

The main endpoint: any status in the request is ignored.
kubectl apply / edit can never overwrite what the operator reported, even by accident.

.../memgraphclusters/my-cluster/status

Only status is taken; everything else is ignored. Even a buggy operator status write cannot change the spec.

generation counts spec changes only
Status writes don't bump metadata.generation. So our status patches don't look like new user intent, and observedGeneration can tell whether status describes the current spec.

```
# config/rbac/role.yaml (operator)
- resources: [memgraphclusters]
  verbs: [get, list, watch]
- resources: [memgraphclusters/status]
  verbs: [get, patch]
```

RBAC grants the halves separately: the operator can't modify the spec at all.

$ Conditions: status users care about

Higher-level status reporting from a controller. Tools and other controllers read a summary without understanding resource-specific details: a Deployment's Available instead of readyReplicas, replicas and the rest.

Ours today

Converged the observed topology matches the spec

Updated every pod runs the current pod template

LicenseApplied every ready pod runs the license in the Secret

Ready
The summary: a MAIN serves **and** the three above are True. Otherwise it carries the first failing reason.

Polarity

Normal-true: Ready, Converged

Normal-false: MemoryExhausted, OutOfDisk

Ideas, not built yet

MemoryExhausted → bring up a new instance, shard.

OutOfDisk → surface it before writes fail.

```
kubectl wait –for=condition=Converged mgc/my-cluster
```

$ Anatomy of a condition: reason and message

```
status:
  conditions:
  - type: Converged
    status: "False"
    reason: RegistrationInProgress
    message: Issuing 2 registration
      command(s) to converge the cluster
    observedGeneration: 3
    lastTransitionTime: "2026-10-13T09:12:04Z"
```

reason

One word, CamelCase: the category of the cause of the current status. Stable, for machines. 27 in our API.

message

A human-readable phrase or sentence. May carry the details of this specific occurrence: counts, names.

More of ours: WorkloadsNotReady, NoCoordinatorLeader, VolumeExpansionRefused, NoCaughtUpSurvivor, LicenseRejected

$ API rule 1: defaults live in the API

Default values are explicit in the API, rather than asserting that unspecified fields get some default behavior.

Avoid: default hidden in Go

```
// +optional
Coordinators *int32 `json:"coordinators,omitempty"`

// somewhere in the operator:
if c.Spec.Coordinators == nil { n = 3 }
```

Do: default in the schema

```
// +kubebuilder:default=3
// +optional
Coordinators *int32 `json:"coordinators,omitempty"`
```

Nested blocks: +kubebuilder:default={} so their inner defaults apply too.

Visible

kubectl get -o yaml shows coordinators: 3.

Written once

The API server fills it in on create; it’s stored.

No silent changes

A new binary can’t change what a running cluster gets.

$ Why: the cost of Go-only defaults

Go-only default

```
$ kubectl get mgc -o yaml
spec:
  dataInstances: 2
  # coordinators? set? exists? 3?
```

Default in the schema

```
$ kubectl get mgc -o yaml
spec:
  coordinators: 3
  dataInstances: 2
```

1 Users can’t see the real value

The YAML doesn’t show the full schema. Is it set? Does the field even exist? People have to guess.

2 One place, no side effects

A schema default is declared in exactly one place. The value can’t be a side effect of some code path.

3 Every client must copy it

Any other client reading an empty field has to guess, or port our default logic. It wouldn’t stay in the operator.

For the operator alone it’s manageable: we could expose the value, the user changes it, we reconcile. The cost lands on everyone else.

$ API rule 2: every field is +required or +optional

```
type MonitoringSpec struct {
  // +optional
  VMAgentRemote *VMAgentRemoteSpec `json:"vmagentRemote,omitempty"`
}

type VMAgentRemoteSpec struct {
  // +required
  RemoteWrite RemoteWriteSpec `json:"remoteWrite"`
}

type RemoteWriteSpec struct {
  // +required
  URL string `json:"url"`
}
```

```
monitoring:
  vmagentRemote:     # optional
    remoteWrite:     # required once written
      url: http://.. # required
```

+optional

May be left out, usually with a default (rule 1).

+required

If the enclosing block is written, this field must be too. The API server rejects it otherwise.

No unmarked fields. Every field states its intent, so the CRD schema, not the Go code, says what a user must write.

$ API rule 3: zero values pass validation

"", 0, [] and {} are valid by default: the schema can't know whether they mean something. The API author has to decide.

No min markers: accepted

```
  awsCredentials:
    secretRef: {name: ""}  # +required -> OK
  dataInstances: 0   # an int -> OK
```

+required only checks the key is there. The empty value still gets in.

With min markers: rejected

```
// +kubebuilder:validation:MinLength=1
// +required
Name string `json:"name"`

// +kubebuilder:validation:Minimum=1
// +optional
DataInstances *int32 `json:"dataInstances,omitempty"`
```

Strings
MinLength=1
18 in our API

Integers
Minimum=N
10 in our API

Slices
MinItems=1
not used yet

Maps
MinProperties=1
not used yet

$ cmd/main.go: the operator's main()

1. **Register types into the scheme**

   client-go built-ins, memgraph.com/v1alpha1, Gateway API, Prometheus monitoring/v1

2. **Configure metrics and health probes**

   secure metrics server, /healthz and /readyz

3. **Create the manager**

   cache, client, leader election

4. **Wire the reconciler**

   `MemgraphClusterReconciler.SetupWithManager(mgr)`

5. **mgr.Start()**

   run until the process gets a signal

Think C++ main()

Set up, wire, run. The operator is basically one reconciler, and main.go plugs MemgraphClusterReconciler into the manager.

$ Who is the operator? Identity and permissions

ServiceAccount
the identity: who is asking

RoleBinding / ClusterRoleBinding
the glue: grants a role to a subject

Role / ClusterRole
the permissions: verbs on resources

Authentication: who are you?
The pod runs as `serviceAccountName: controller-manager`. Kubernetes mounts its token into the pod; client-go's in-cluster config sends it with every request, and the API server sees `system:serviceaccount:<ns>:controller-manager`.

Authorization: may you?
RBAC checks every request: verb + resource (+ subresource) + namespace, against the roles bound to that identity. Denied by default; rules only ever add permissions.

`Role` applies inside one namespace. `ClusterRole` applies to the whole cluster, and also covers cluster-scoped resources and URLs like `/metrics`.

$ Our RBAC: which roles, and why

ClusterRole + ClusterRoleBinding manager-role	MemgraphClusters can live in any namespace, so the operator needs StatefulSets, Services, ConfigMaps, Pods, PVCs, Secrets (read only) and CRs cluster-wide. Generated from +kubebuilder:rbac markers.
Role + RoleBinding leader-election-role	The leader-election Lease (plus ConfigMaps and Events) lives only in the operator's own namespace. No reason to grant it anywhere else.
ClusterRole + ClusterRoleBinding metrics-auth-role	Protects /metrics: create TokenReviews and SubjectAccessReviews to check who is scraping. Both are cluster-scoped APIs.

Why a dedicated ServiceAccount? Every namespace has a "default" one, and every pod that names none runs as it. Binding our roles to default would hand cluster-wide rights (Secrets included) to every pod there.

All bound to controller-manager only. Also shipped, not used by the operator: admin / editor / viewer and metrics-reader ClusterRoles.

$ Reconcile(): a key, not a diff

```
func (r *MemgraphClusterReconciler) Reconcile(
    ctx context.Context, req ctrl.Request,
) (ctrl.Result, error)
```

What it gets

ctx cancellation, logger

req just namespace/name of a MemgraphCluster

What it does not get

~~old object~~
~~new object~~
~~what changed~~

No input/output beyond the key.

$ One reconcile pass

1 **Look up the key**

r.Get(ctx, req.NamespacedName, &cluster) hits the cache, in the MemgraphCluster informer

2 **Retrieve the object**

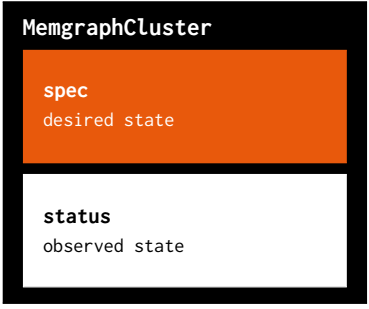
spec = desired state the user wrote; status = what the operator last observed

3 **Compare with the world**

StatefulSets, pods, volumes, Memgraph itself

4 **Converge**

create, patch or restart until reality matches the spec, then update status



$ Fetching the cluster: from the cache, maybe stale

```
var cluster memgraphcomv1alpha1.MemgraphCluster
if err := r.Get(ctx, req.NamespacedName, &cluster); err != nil {
    return ctrl.Result{}, client.IgnoreNotFound(err)
}
```

r.Get reads the cache, not the API server. NotFound means the cluster was deleted: nothing to do.

Stale spec: just a delay

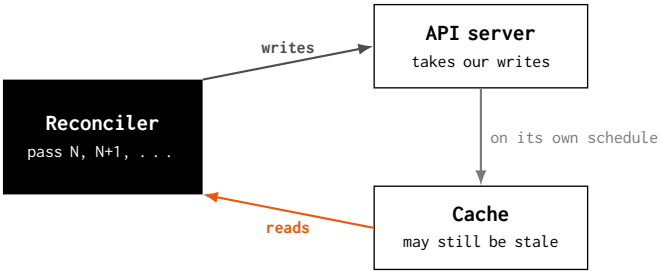
We may act on the old spec for one pass. When the cache catches up, its update event enqueues the cluster again.

Stale status: briefly wrong

Users could see an incorrect value for a moment. The next reconcile reads fresh state and writes it right.

Level-triggered pays off: staleness costs time, not correctness.

$ No read-your-own-writes



StatefulSet generations

Updated in pass N; pass N+1 still reads the older cached version.

Pod deletions

Deleted in pass N; pass N+1 still sees the pod in the cache and must not act on it.

Our own write may not be in the cache yet. Usually that just delays us. When the next pass acts on what it just wrote, a stale read can lead to a wrong action.

$ Two fixes: APIReader or expectations

APIReader

Read straight from the API server instead of the cache.

+ Simple: no extra state to track
+ Few reads actually need it
— More network traffic and API server load

We already use it for Secret data.

Expectations

Remember what we wrote; next pass checks the cache has caught up.

+ Reads stay cached
+ A standard Kubernetes controller pattern
— In-memory state to keep correct
— Not caught up: skip the pass, requeue

What we use, for StatefulSet scale-down

$ Expectations in practice: scaling 3 → 5 → 3

1 Grow to 5

desired = max(spec, cached). Pods 4 and 5 are created. Safe even with a stale cache.

2 Shrink to 3

We write 3 replicas. The max no longer protects us.

3 Next pass

The cache may still show 5 pods. Acting on it would be wrong.

internal/expectations

Expect	On shrink, record the StatefulSet's UID and the generation our write produced.
Satisfied	Next pass: is the cached StatefulSet at least that new?
Not yet	Skip the pass, requeue after 10 s; meanwhile the cache catches up.

$ Optimistic concurrency with resourceVersion

etcd is MVCC: every write produces a new version. The API server exposes it as metadata.resourceVersion.



Ours: the PVC resize patch uses MergeFromWithOptimisticLock: it sends the resourceVersion of the PVC it read, so a stale read ends in 409 instead of overwriting a newer object.

$ Stale reads: three ways out

resourceVersion is opaque: never compare two of them, nothing guarantees they increase. **generation** is a monotonically increasing integer for the spec, so “is the cache at least as new as my write?” can be answered. That’s what expectations record.

OURS

Expectations

Record the generation we wrote; skip and requeue until the cached object is at least that new.

Cost: in-memory state

CONSIDERED

APIReader

Read straight from the API server: always fresh.

Cost: a network round trip on every pass

PVC RESIZE

resourceVersion as a lock

Send the cached resourceVersion with the write; stale → 409 → requeue. Our PVC resize patch already works this way.

Cost: a failed write and a requeue whenever the cache is stale

$ The reconcile loop

```
func (r *MemgraphClusterReconciler) Reconcile(
    ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
  // 1 fetch
  cluster := new(memgraphcomv1alpha1.MemgraphCluster)
  if err := r.Get(ctx, req.NamespacedName, cluster); err != nil {
    return ctrl.Result{}, client.IgnoreNotFound(err)
  }
  // 2 snapshot
  orig := cluster.DeepCopy()
  cluster.InitializeConditions()
  // 3 do the work
  result, reconcileErr := r.reconcileCluster(ctx, cluster)
  // 4 skip no-op writes
  if equality.Semantic.DeepEqual(orig.Status, cluster.Status) {
    return result, reconcileErr
  }
  // 5 patch the status
  if err := r.Status().Patch(ctx, cluster,
      client.MergeFrom(orig)); err != nil {
    return ctrl.Result{}, errors.Join(reconcileErr,
      fmt.Errorf("patching MemgraphCluster status: %w", err))
  }
  return result, reconcileErr
}
```

1 Fetch

From the cache. NotFound = deleted, done.

2 Snapshot

orig is the baseline for the status diff; conditions get initialised.

3 Do the work

reconcileCluster converges the world and updates cluster.Status in memory.

4 Skip no-op writes

Status unchanged: no API call, no extra event.

5 Patch the status

MergeFrom(orig) sends only the diff. Errors are joined; an error means requeue with backoff.

$ Three ways out of Reconcile

1 Error

```
return ctrl.Result{}, err
```

controller-runtime requeues with exponential backoff, so a failing cluster can't saturate the operator.

2 RequeueAfter

```
ctrl.Result{RequeueAfter: d}, nil
```

Not fatal, check again in d. We use 10 s while Memgraph is unavailable: pods not ready, coordinator unreachable, no survivor.

3 Requeue

```
ctrl.Result{Requeue: true}, nil
```

Requeue now, through the rate limiter; you can't pick the delay. We don't use it.

Nothing happening? A converged cluster still requeues after 30 s, because a pod can lose its registration without producing any watch event.

$ Sync period, priority queue, and a fleet

10 h

Sync period (our default)

Every cached object is redelivered as a fake update: a safety net for missed events and leader handovers. We run one replica, so we leave it.

Work queue is now a priority queue

real event · 0 | fake update · −100 | initial list · −100

served first → last

Fake updates (unchanged resourceVersion) and the initial list go in at low priority automatically, so real events from other clusters jump ahead. A requeue can set its own priority (Result.Priority).



One operator, a fleet of clusters

No need for one operator per cluster: a single operator manages as many MemgraphClusters as needed. Fleet scale will likely be a production requirement later; it isn't tuned for that yet.

$ Two “sync periods”: not the same thing

controller-runtime SyncPeriod

cache.Options · default 10h (±10% jitter)

- Where: in our operator, one cache per process.
- What: every object already in the cache is replayed as an update (same resourceVersion), so every MemgraphCluster goes through Reconcile.
- No API calls: it replays the cache, it doesn’t re-list.
- Why: a safety net, so nothing a missed event or our own logic dropped stays unconverged.

kubelet syncFrequency

KubeletConfiguration · default 1m (×1.0 to 1.5)

- Where: on every node, one kubelet each.
- What: reruns syncPod for every pod on that node: containers, probes, and Secret / ConfigMap volumes.
- Nothing to do with our operator or its cache.
- Why: node-level convergence; it’s what eventually delivers updated Secret files (see the TLS section).

Same idea, different layer: both are periodic level-triggered resyncs. One replays our cache into Reconcile every ∼10h; the other re-syncs each pod on a node every 1–1.5min.

$ Concurrent reconciling: four workers

1 Same key, never twice

The work queue never hands the same ns/name to two workers at once. One cluster's passes stay serialized, whatever the setting.

2 Shared state is safe

The cache and clients are already safe for concurrent use, and so is the rest of the reconciler's shared state. No consistency issues.

3 Why 4

A pass can spend tens of seconds dialling pods that don't answer. More workers keep clusters from queueing behind each other.

How it's set

In SetupWithManager. Tune it further once one operator runs many clusters and metrics say so:

```
WithOptions(controller.Options{
MaxConcurrentReconciles: 4})
```

Metrics to watch first

workqueue_depth
keys waiting for a worker

controller_runtime_reconcile_time_seconds
how long each pass takes

$ Inside client-go's workqueue

```
type Type struct {
  // order of work; every item here
  // is dirty and not processing
  queue []t
  // everything that needs processing
  dirty set
  // being processed right now; may
  // also be dirty again
  processing set
  cond *sync.Cond
  ...
}
```

Add(key)

Already dirty? Drop it (coalesce). Otherwise mark dirty; if it isn't being processed, append to queue and Signal().

Get()

Wait() until the queue isn't empty. Pop the key, move it from dirty to processing.

Done(key)

Remove from processing. Dirty again meanwhile? Re-queue it and Signal().

A key in processing is never handed out again until Done(): one cluster, one worker at a time.

$ Using client-go's workqueue: a minimal example

```
func fillQueue(queue workqueue.Interface) {
  queue.Add("this")
  queue.Add("is")
  // ... "a", "complete", "sentence"
  queue.ShutDown()
}

func readFromQueue(queue workqueue.Interface, stop chan int) {
  for {
    item, shutdown := queue.Get()
    if shutdown {
      stop <- -1
      return
    }
    fmt.Printf("Got item: %s\n", item)
    queue.Done(item)
  }
}

func main() {
  stop := make(chan int)
  myQueue := workqueue.New()
  go fillQueue(myQueue)
  go readFromQueue(myQueue, stop)
  <-stop
}
```

workqueue.New()
A plain queue: dirty, processing, queue and a sync.Cond.

Add + ShutDown
The producer adds keys, then shuts down; queued keys still drain.

Get() blocks
cond.Wait() until there's an item, or shutdown = true.

Done(item)
Always call it: until then the key stays in processing.

christianb93/kubernetes-client-examples, example3/main.go (trimmed: sleeps and prints)

$ Go’s condition variable: sync.Cond

```
package main

import "sync"

type SharedQueue struct {
  mu    sync.Mutex
  cond  *sync.Cond
  items []int
}

func NewQueue() *SharedQueue {
  q := &SharedQueue{}
  q.cond = sync.NewCond(&q.mu)
  return q
}
```

Wait(): unlock, sleep, re-lock on wake-up.

for, not if: re-check after every wake-up.

```
// Consumer: waits until data is available
func (q *SharedQueue) Pop() int {
  q.mu.Lock()
  defer q.mu.Unlock()

  // Always use a for loop to check the condition
  for len(q.items) == 0 {
    q.cond.Wait() // Unlocks q.mu, sleeps, re-locks q.mu on wake-up
  }

  item := q.items[0]
  q.items = q.items[1:]
  return item
}

// Producer: pushes data and signals waiting consumers
func (q *SharedQueue) Push(val int) {
  q.mu.Lock()
  defer q.mu.Unlock()

  q.items = append(q.items, val)

  // Wake up one consumer (or use q.cond.Broadcast() for all)
  q.cond.Signal()
}
```

$ sync.Cond vs std::condition_variable

	Go	C++
Lock	`sync.Mutex`	`std::mutex`
Condition variable	`sync.NewCond(&mu)`	`std::condition_variable`
Wait	`cond.Wait()`	`cv.wait(lock)`
Wake one	`cond.Signal()`	`cv.notify_one()`
Wake all	`cond.Broadcast()`	`cv.notify_all()`
Spurious wake-ups	`for !ready { cond.Wait() }`	`cv.wait(lock, [&]{ return ready; })`

Same model, two differences: Go binds the lock once, in NewCond; C++ passes it to every wait(). Go's Wait() has no timeout (use channels or a context), C++ has wait_for / wait_until.

$ How the operator sees the world

1

Cached client

Reads from the local informer cache, writes go to the API server.

2

APIReader

Uncached reads from the API server, e.g. Secret data the cache does not hold.

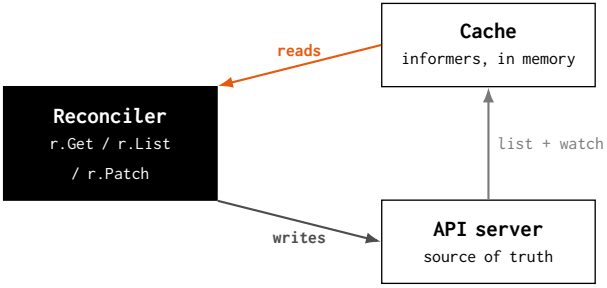
3

Bolt connection

Talks to Memgraph itself. If a server deadlocks, the connection is dropped; the operator doesn't hang.

30 s timeout

$ Cached client: read local, write remote



Scoped by label

Pods, ConfigMaps and Deployments are cached only if managed by the operator; PVCs only if labelled as Memgraph's. Memory stays bounded.

Reads are cheap and fast; the cache may lag the API server by a moment.

$ Informers: one local copy per kind

1 **Keeps a local copy**
of one kind's objects: MemgraphCluster, Pod, StatefulSet. . .

2 **Stays in sync**
with the API server through list + watch

3 **Notifies**
interested event handlers when something changes

A kind gets an informer in one of two ways

1 **A watch is set up**
For / Owns / Watches in SetupWithManager

2 **The client reads it**
the first cached Get/List starts it on demand. We no longer rely on this.

$ Under the informer: list, then watch

```
$ kubectl proxy &
$ curl "localhost:8001/apis/memgraph.com/v1alpha1/namespaces/memgraph/\
    memgraphclusters?watch=true&resourceVersion=48213"
{"type":"MODIFIED","object":{"kind":"MemgraphCluster",
  "metadata":{"name":"my-cluster","resourceVersion":"48297",...}}}
{"type":"ADDED","object":{...}}
...   # the connection stays open, events keep streaming
```

LIST
all objects + the list's resourceVersion

WATCH
?watch=true from that resourceVersion

Store
apply ADDED / MODIFIED / DELETED

Handlers
call what the controller registered

The controller registers functions with the informer, called whenever a resource changes.

And reads the store for the current state, then acts to drive the system to the target state.

$ The “store” is a cache.Indexer

```
// k8s.io/client-go/tools/cache
type Store interface {
    Add(obj interface{}) error
    Update(obj interface{}) error
    Delete(obj interface{}) error
    List() []interface{}
    ListKeys() []string
    Get(obj interface{}) (item interface{},
        exists bool, err error)
    GetByKey(key string) (item interface{},
        exists bool, err error)
    Replace([]interface{}, string) error
    Resync() error
}
```

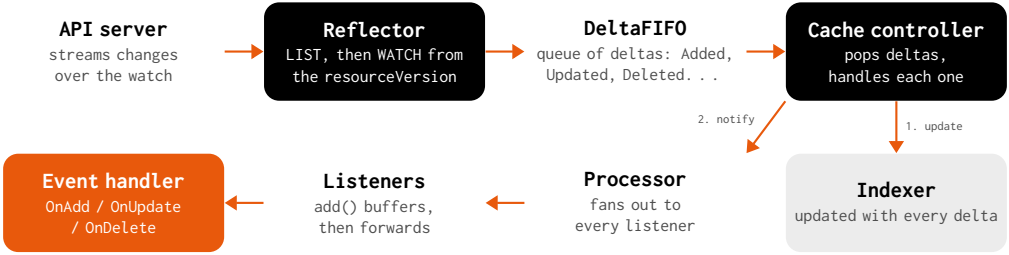
```
type Indexer interface {
    Store  // embeds every method on the left
    ByIndex(indexName, indexedValue string)
        ([]interface{}, error)
    IndexKeys(...) / Index(...)
    AddIndexers(Indexers) error
}
```

The informer’s cache is an Indexer

cache.NewIndexer() returns the cache struct: a thread-safe map keyed by “namespace/name”, plus secondary indexes.

Writes: only the informer (Add/Update/Delete from WATCH, Replace from LIST). **Reads:** the controller, via Get / GetByKey / List / ByIndex.

$ Inside an informer: the components



The handler functions are methods of our controller.
They get the object (and the old one on update), can use the freshly updated indexer to look up related objects, and act to bring the cluster to the desired state.

$ Listeners and handlers

```
// k8s.io/client-go/tools/cache
type Delta struct {
    Type   DeltaType  // Added, Updated, Deleted,
                      // Replaced, Sync
    Object interface{}
}

type ResourceEventHandler interface {
    OnAdd(obj interface{}, isInInitialList bool)
    OnUpdate(oldObj, newObj interface{})
    OnDelete(obj interface{})
}
```

```
// processorListener, simplified
func (p *processorListener) add(n any) {
    p.addCh <- n
}
// pop(): addCh -> ring buffer -> nextCh
// run(): for n := range p.nextCh {
//     call OnAdd / OnUpdate / OnDelete
// }
```

Buffered per listener: a slow handler never blocks the indexer or other handlers.

Keep handlers cheap. In practice they only enqueue a key (enqueueFoo, or controller-runtime's EnqueueRequestForObject). The worker then reads the indexer and does the real work in Reconcile.

$ Why informers and a cache?

Without a cache

- Every reconcile GETs and LISTs straight from the API server
- More clusters × objects × reconciles = a flood of requests
- A controller could DDoS its own API server

→

With an informer

- One watch per kind against the API server
- Every add, update and delete streams in
- A complete in-memory copy; reads never leave the process

Steady state takes the fast path. When desired = current, every read comes from the cache; no slow routes.

Work queue coalesces



5 changes, 1 reconcile

Pool of workers

As many as needed. We run 4 (MaxConcurrentReconciles = 4).

$ Watches: which one, and why

All are an informer plus a handler. They differ in how an event finds its cluster, and what is cached.

```
ctrl.NewControllerManagedBy(mgr).
  For(&MemgraphCluster{}).
  Owns(&appsv1.StatefulSet{}).
  Owns(&appsv1.Deployment{}).
  Owns(&corev1.Service{}).
  Owns(&corev1.ConfigMap{}).
  WatchesMetadata(&corev1.Secret{},
    ...clustersForSecret).
  Watches(&corev1.PersistentVolumeClaim{},
    ...clusterForClaim).
  Watches(&corev1.Pod{},
    ...clusterForPod).
  Named("memgraphcluster").Complete(r)
```

For **The object is the cluster**

MemgraphCluster: enqueues its own ns/name

Owns **We created it and set ourselves as owner**

StatefulSet, Deployment, Service, ConfigMap, Gateway API, ServiceMonitor: follows ownerReference (+ self-healing, GC)

Watches **Not ours, but we can derive the cluster**

PVC, Pod: made by the StatefulSet controller; labels name the cluster

WatchesMetadata **Like Watches, without the data**

Secret: user's own; map to clusters whose spec names it

$ Owner references: the link back to the parent

```
# on every child object we apply
metadata:
  ownerReferences:
  - apiVersion: memgraph.com/v1alpha1
    kind: MemgraphCluster
    name: my-cluster
    uid: 3f6c...
    controller: true
    blockOwnerDeletion: true
```

stamped by controllerutil.SetControllerReference before every apply

1 Garbage collection

Delete the MemgraphCluster and the garbage collector deletes every child pointing at it, and their children in turn. The operator never deletes its workloads itself.

2 Events flow up

Owns(&StatefulSet{}): when a child changes, its controller owner is enqueued for reconcile. No map function needed.

Our tree: MemgraphCluster → StatefulSets, Deployment, Services, ConfigMaps (ours) → Pods, PVCs (stamped by the StatefulSet controller). Pods are grandchildren, so we map them to the cluster by label instead.

$ Finalizers: referential integrity for storage

A finalizer is a key in metadata.finalizers. Deleting the object only sets deletionTimestamp; it stays until every finalizer is removed.



PVC can't go while a pod still uses it.

pvc-protection is removed only when no pod mounts the claim.

PV can't go while a claim is bound to it.

pv-protection is removed only when no PVC is bound.

Like a foreign key: storage can't be pulled from under a running Memgraph. Kubernetes adds both; the operator owns no finalizer of its own.

$ Pods: a watch set up front

```
Watches(&corev1.Pod{}, handler.EnqueueRequestsFromMapFunc(clusterForPod))
```

1 Starts with the manager

The Pod informer starts and syncs with the manager, so no reconcile ever blocks on it.

2 Only Memgraph pods

No other pod in the cluster is ever looked at or cached.

3 Mapped by label

Each pod maps to its MemgraphCluster through its labels.

4 Fails early

Problems such as missing RBAC show up at startup, not mid-reconcile.

$ PVC and Secret changes: mapped, never delivered



The reconciler never finds out the PVC or Secret changed. It only gets the cluster's name.

PVC · clusterForClaim

Monitors every PVC labelled app.kubernetes.io/name=memgraph, configured in main.go:

```
&corev1.PersistentVolumeClaim{}: {Label: memgraphClaims}
```

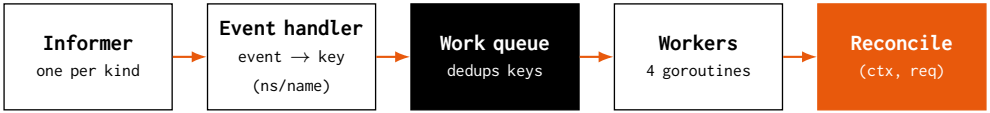
Maps to the cluster named in its app.kubernetes.io/instance label.

Secret · clustersForSecret

Monitors the metadata of every Secret; a data edit still bumps resourceVersion, so it is seen.

Maps to every MemgraphCluster in that namespace whose spec names the Secret as its license or AWS credentials Secret.

$ From an event to Reconcile()



Only a key travels, never the object. Ten events for the same cluster collapse into one queue entry, and the reconciler re-reads current state from the cache. That makes reconciliation level-based: it acts on what is, not on what happened.

$ The event handler, by hand: object → key

```
fooInformer.Informer().AddEventHandler(
  cache.ResourceEventHandlerFuncs{
    AddFunc: controller.enqueueFoo,
    UpdateFunc: func(old, new interface{}) {
      controller.enqueueFoo(new)
    },
  })
```

```
func (c *Controller) enqueueFoo(obj interface{}) {
  if objectRef, err := cache.ObjectToName(obj); err != nil {
    utilruntime.HandleError(err)
    return
  } else {
    c.workqueue.Add(objectRef)
  }
}
```

kubernetes/sample-controller, controller.go

Invoked by the informer
On every add or update of a Foo, the informer calls enqueueFoo with the object.

Object → key
cache.ObjectToName keeps only namespace/name. The object never reaches the queue.

Add(key)
The queue dedups: a key already waiting is not added twice.

controller-runtime does this for us
For(&MemgraphCluster{}) installs the same handler (EnqueueRequestForObject).

$ Edge-triggered vs level-triggered

Edge-triggered

Act on the transition

```
replicas: 1 -> 3
=> add 2
```

Fragile in a distributed system: crashes, unstable networks and different latencies lose or reorder events, and the diff is wrong.

Level-triggered

Act on the current state

```
want 3, have N
=> converge to 3
```

No matter how we got here: a diff of 2, 5 or 10, missed events or a restart. Read the state, act on it.

Kubernetes controllers combine both. The event (edge) only says that reconcile should run. What to do comes from the current state (level).

$ Manager and controller

Manager

Process-level container for everything shared:

- Client
- **Cache**
- Leader election
- Metrics / health
- Controllers

Cache = set of informers, one per GVK (kind)

Controller

> Owns a work queue

> 4 worker goroutines take keys off the queue

> Dispatches each one to the reconciler: MemgraphClusterReconciler

$ Why controller-runtime: the NFD incident

node-feature-discovery (NFD)

Labels nodes with the hardware features it finds on bare metal. Scheduling (e.g. GPUs) depends on those labels.

- v0.16: ∼4,000 nodes, a ∼140 KB NodeFeature object each. The controller's LIST timed out, so its informer cache never synced.
- It used client-go directly and didn't check WaitForCacheSync(), so it ran on an empty cache.
- Missing from the cache was read as "no features": nearly all labels were removed at once, and pods could no longer schedule.

```
// client-go: lifecycle is your job
factory.Start(ctx.Done())
if !cache.WaitForCacheSync(ctx.Done(),
        informer.HasSynced) {
    return errors.New("caches not synced")
}  // NFD skipped this check
```

controller-runtime does it for us. The manager starts our controller only after its caches have synced. If they don't within CacheSyncTimeout (2 min default), start fails instead of reconciling on an empty cache.

Missing from the cache ≠ missing from the cluster. Never act on a cache that hasn't synced. The framework makes the safe path the default; it doesn't replace scale testing.

$ ls talk/

$ cd Demo and operator vs Helm/ 

$ Demo

```
# install the operator
$ helm repo add memgraph https://memgraph.github.io/helm-charts
$ helm install memgraph-operator memgraph/memgraph-operator

# create a cluster and watch it converge
$ kubectl apply -f cluster.yaml          # 3 coordinators, 2 data
$ kubectl get mgc -w

# scale out: change a number
$ kubectl patch mgc my-cluster --type=merge -p '{"spec":{"dataInstances":3}}'

# a run-time flag: no restart
$ kubectl patch mgc my-cluster --type=merge \
    -p '{"spec":{"flags":{"data":{"log-level":"DEBUG"}}}}'
```

$ What's new compared to the Helm charts

Scaling
change a number; the operator adds pods and registers them

Layout
one StatefulSet per role, pods indexed 0. . . n-1

License
edit the Secret; applied live with SET DATABASE SETTING

Storage
one claim per pod for data + logs; logs optional; online resize

Probes
readiness only: no startup or liveness probe

Security
securityContext block (uid / gid / fsGroup), OpenShift-friendly

Scheduling
one anti-affinity rule: type × scope × topologyKey

Ports
fixed inside the cluster: 7687, 10000, 20000, 12000

Flags
one ConfigMap per role; run-time flags applied live, no restart

External access
two modes: LoadBalancer or Gateway

$ Scaling: change a number

Helm charts
A StatefulSet per instance: memgraph-coordinator-1, -2, -3, memgraph-data-0, -1, each with one pod. Growing the cluster meant new values, new objects and registering by hand or via the setup job.

Operator
One StatefulSet per role: <cluster>-coordinator and <cluster>-data, pods indexed 0. . . n-1. The replica count is the spec's number.

```
spec:
  coordinators: 3
  dataInstances: 3   # was 2
```

New pod
StatefulSet adds data-2

Register
REGISTER INSTANCE on the leader

Connected
replicates from MAIN

Down works too: members above the new count are retired first (MAIN moved off, UNREGISTER INSTANCE; REMOVE COORDINATOR, after YIELD LEADERSHIP if needed). Coordinators stay odd and ≥ 3.

$ License: edit the Secret, applied live

```
spec:
  secrets:
    name: memgraph-secrets  # default; any name
    licenseKey: MEMGRAPH_ENTERPRISE_LICENSE
    organizationKey: MEMGRAPH_ORGANIZATION_NAME
```

Same as the Helm charts: the license lives in a Secret you own; the CR only references it.

You update the Secret
new license / organization name

Operator notices
metadata watch; value read uncached

SET DATABASE SETTING
'organization.name', 'enterprise.license' on every ready pod, only where it differs

LicenseApplied=True
a SettingApplied Event per pod; no restart

And the env variables? A pod reads the Secret into its environment only when it starts, so running pods keep the old values until their next restart. That's fine: Memgraph already runs the new license from the SET, and a restarted pod starts with the new Secret.

$ Storage: one claim, logs optional

```
/var/lib/memgraph/       # lib claim, per pod
|-- mg_data/             # --data-directory
`-- logs/memgraph.log    # with logFile: LibClaim
```

Helm: a lib claim and a separate log claim per instance.

logFile: LibClaim (default)
Log files on the same claim, counted in libPVCSize.

logFile: None
No log file: stderr only, shipped by Vector to VictoriaLogs (like our CI). What many users pick.

Resizing: raise libPVCSize and the operator grows every claim under the running pods, if the StorageClass allows expansion. No restart on drivers that grow volumes in use. No separate log disk yet; we'll add it if people ask.

$ Probes: readiness only

Helm charts
Startup + liveness + readiness. The startup probe allowed 2 hours for recovery. Large datasets took longer, the kubelet killed the pod, recovery started over: CrashLoopBackOff.

Operator
Only a TCP readiness probe (Bolt for data, coordinator port for coordinators). A recovering pod is just unready: no traffic, no restarts, however long it takes.

Why it's safe: Memgraph opens Bolt only after recovery, so during recovery a slow pod and a hung pod look the same to a probe. If Memgraph dies, the process exits and the container restarts anyway. Restarts of a healthy instance belong to the operator's rolling restart. Other operators skip them too; if users need them, we'll add them (still v1alpha1).

$ Security context and node setup

```
spec:
  securityContext:    # absent: 101/103/103
    runAsUser: 1000680000
    runAsGroup: 1000680000
    fsGroup: 1000680000
  fixOwnershipInitContainer: {}
  sysctlInitContainer:
    maxMapCount: 524288
```

One block instead of loose ids
Helm had memgraphUserId / memgraphGroupId. Now runAsUser, runAsGroup, fsGroup; an empty block writes none, so OpenShift's restricted-v2 can assign them. Not tested on OpenShift yet.

Python modules
Since Memgraph 3.13/3.14, installed globally: query modules' pip packages work with any uid.

Core dumps
Off by default. Per role, with an optional uploader sidecar.

Node tuning
sysctl init container for vm.max_map_count and fix-ownership container: same as before, opt-in.

$ Scheduling: same features, better organized

```
spec:
  scheduling:
    podAntiAffinity:
      type: required      # or preferred
      scope: cluster      # or role
      topologyKey: topology.kubernetes.io/zone
    data:
      nodeSelector:
        agentpool: memgraphdata
      tolerations: [...]
```

type
preferred: try to keep pods apart · required: stay Pending if it can't

scope
role: coordinators apart, data apart · cluster: every pod apart

topologyKey
kubernetes.io/hostname (nodes) or a zone label

Helm's modes map onto it: default = preferred + role, parity = required + role, unique = required + cluster. Per role: nodeSelector, tolerations, topologySpreadConstraints, priorityClassName. Example: data instances on their own node pool with more memory and a storage class that can resize.

$ Flags: one ConfigMap per role, live when possible

```
spec:
  flags:
    coordinators:
      log-level: INFO
    data:
      query-execution-timeout-sec: "120"  # run-time
      storage-gc-cycle-sec: "60"          # startup
```

Helm: flags per instance, on the command line. Now one flag file per role, <cluster>-data-flags and <cluster>-coordinator-flags, loaded with –flag-file.

Run-time flag
The operator knows which flags are database settings. It updates the ConfigMap and runs SET DATABASE SETTING on every ready pod. No restart, and the next start reads the same value.

Startup flag
The ConfigMap changes and so does a hash on the pod template, so the operator rolls the pods right away: data instances first, MAIN last, the Raft leader last.

$ External access: two modes

type: LoadBalancer
One LoadBalancer Service shared by all coordinators, and one per data instance (a Service per pod).

type: Gateway
One Gateway the operator creates: a TCPRoute for the coordinators and one per data instance, each on its own port.

DNS
With external-dns, annotate hostnames (data-{ordinal}.memgraph.example.com): records are created and members registered at those names. Without it, members are registered at the IP or hostname the cloud reports; DNS names are yours to set up.

Registration is automatic
The operator announces the external Bolt address in the routing table and runs UPDATE CONFIG when it changes. No setup commands by hand.

The rule we recommend: coordinators behind one address, every data instance behind its own. Fewer options than the charts, the ones setups converged on.

$ Secret volumes: files don’t change right away

Common belief
Update the Secret, and the mounted files change immediately.

What happens
The kubelet already has the new data, but nothing tells the volume code to rerun. Files change only when the pod is synced.

When does the kubelet sync a pod?

1 **Something about the pod changes**
Pod object updated, container lifecycle event. . .
Rare while a pod just runs.

2 **Periodic resync**
Every syncFrequency (60 s) × jitter 1.0–1.5, so 60–90 s. This is what delivers Secret and ConfigMap updates.

Never updated: subPath mounts and environment variables. **Shortcut:** touching a pod annotation requeues the pod at once (no restart), but nobody edits pods by hand.

$ Renewing a certificate on a running cluster

Secret updated in place
cert-manager, External Secrets, Vault or kubectl apply

kubelet swaps the files
in /etc/memgraph/ssl on every pod, within ~60–90 s

Memgraph reloads
RELOAD BOLT SERVER TLS on every instance: manual today

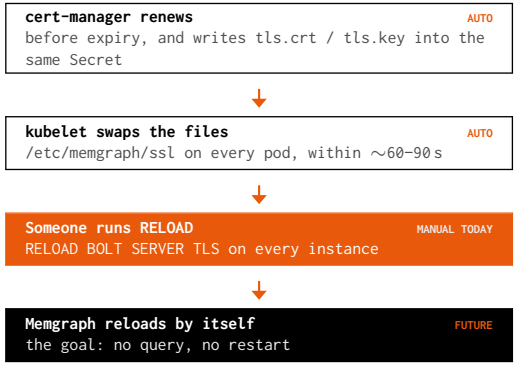
Who updates the Secret?
Manually: kubectl apply a new kubernetes.io/tls Secret under the same name. Automatically: a cert-manager Certificate renews before expiry (by default at 2/3 of its lifetime) and rewrites the same Secret.

What the operator does
Mounts spec.tls Secrets without subPath, so in-place updates reach the pods. It never watches them and restarts nothing. A different secretRef.name is a pod-template change and rolls the cluster.

intraCluster: Raft between coordinators already re-reads the files on every handshake; replication and the management RPC still need RELOAD INTRA_CLUSTER TLS.

$ Rotation with cert-manager, end to end

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: bolt-tls
  namespace: memgraph
spec:
  secretName: bolt-tls  # our tls.bolt Secret
  duration: 2160h       # 90 days
  renewBefore: 360h     # renew 15 days early
  dnsNames:
  - memgraph.example.com
  - "*.my-cluster-data.memgraph.svc.cluster.local"
  issuerRef:
    kind: ClusterIssuer
    name: letsencrypt
```



$ Closing the gap: who issues the reload?

Option A: the operator

Watch the TLS Secrets (metadata only, like other Secrets) and run RELOAD on every instance when one changes.

Catch: the API says when the Secret changed, not when the kubelet swapped the files, up to ~90 s later. The operator would have to retry, e.g. until the certificate a pod serves matches the Secret.

Option B: Memgraph itself

Notice new files and reload on its own, the way Raft already does per handshake.

No timing guesswork, no extra operator logic, and it works outside Kubernetes too. Watch the directory, not the file: the kubelet swaps a `..data` symlink.

Direction: do it in the Memgraph binary. Until then, a renewal on Kubernetes is automatic up to the files, plus one manual query per instance.

$ The rest: mostly as before

Ports Helm: configurable
fixed: 7687 / 10000 / 20000 / 12000

TLS see TLS slides
Bolt TLS and intra-cluster mutual TLS

clusterDomain same
cluster.local by default

resources same
requests and limits per role

labels same
pod, StatefulSet, Service labels per role

extraEnv same idea
per role, literal values only

extraVolumes same
extra volumes and volume mounts

containers same
user containers and init containers

monitoring same
local and remote, as before

```
$ ls talk/
$ cd Sources/
```

$ Sources (1/8)

1 **Kubernetes controller pitfalls** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/controller-pitfalls/

2 **CRD generation pitfalls** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/crd-generation-pitfalls/

3 **Kubernetes List performance: the deadly combo** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/kubernetes-list-performance/#deadly-combo

4 **Kubernetes evictions** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/kubernetes-evictions/

5 **The OpenAI Kubernetes incident** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/openai-kubernetes-incident/

$ Sources (2/8)

6 **OpenAI incident report** OpenAI status page: the official write-up (see #5)
https://status.openai.com/incidents/ctrsv3lwd797

7 **Tale of a Kubernetes node-feature-discovery incident** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/nfd-incident/

8 **All node labels removed while informer cache failed to sync** NFD GitHub issue #1802
https://github.com/kubernetes-sigs/node-feature-discovery/issues/1802

9 **KWOK: Kubernetes WithOut Kubelet** kubernetes-sigs: simulate thousands of fake nodes for scale testing
https://kwok.sigs.k8s.io/

10 **Kubernetes secret volumes delay** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/kubernetes-secret-volumes-delay/

$ Sources (3/8)

11 **Pitfalls reloading files from Kubernetes Secret & ConfigMap volumes** Ahmet Alp Balkan, ahmet.im
https://ahmet.im/blog/kubernetes-inotify/

12 **AtomicWriter** k8s.io/kubernetes/pkg/volume/util: how the kubelet swaps volume files
https://pkg.go.dev/k8s.io/kubernetes/pkg/volume/util#AtomicWriter

13 **inotify(7): monitoring filesystem events** Linux man-pages, man7.org
https://man7.org/linux/man-pages/man7/inotify.7.html

14 **Things UNIX can do atomically** Richard Crowley, January 2010
https://rcrowley.org/2010/01/06/things-unix-can-do-atomically.html

15 **Kubernetes API conventions** kubernetes/community, sig-architecture
github.com/kubernetes/community/. . . /sig-architecture/api-conventions.md

$ Sources (4/8)

16 **Kubernetes design principles: control logic** kubernetes/design-proposals-archive
github.com/kubernetes/design-proposals-archive/. . . /principles.md#control-logic

17 **The feedback loops behind Kubernetes** Fatih Arslan, PlanetScale blog, June 2026
https://planetscale.com/blog/the-feedback-loops-behind-kubernetes

18 **Expectations in ECK (cloud-on-k8s)** Elastic, expectations.go
github.com/elastic/cloud-on-k8s/. . . /expectations/expectations.go#L16-L78

19 **Unix philosophy: Origin** Wikipedia
https://en.wikipedia.org/wiki/Unix_philosophy#Origin

20 **Learning Concurrent Reconciling** Fei Guo, OpenKruise blog, November 2019
https://openkruise.io/blog/learning-concurrent-reconciling/

$ Sources (5/8)

21 **Operator Architectures Best Practices** operator-framework Google Group thread, January 2021
groups.google.com/g/operator-framework/c/K7zwQiCJVYg/m/03NT2HYeCAAJ

22 **KubeCon + CloudNativeCon Europe 2025 talk** Sched event 1txGQ
https://kccnceu2025.sched.com/event/1txGQ

23 **YouTube talk, from 8:59** youtube.com/watch?v=zCXiXKMqnuE
https://www.youtube.com/watch?v=zCXiXKMqnuE&t=539s

24 **client-go workqueue example** christianb93/kubernetes-client-examples
github.com/christianb93/kubernetes-client-examples/.../example3/main.go

25 **Extending Kubernetes with custom resources and custom controllers** LeftAsExercise, July 2019
leftasexercise.com/2019/07/04/extending-kubernetes-with-custom-resources-and-custom-controllers/

$ Sources (6/8)

26 **Understanding Kubernetes controllers, part I** LeftAsExercise, July 2019: queues and the core controller loop
leftasexercise.com/2019/07/08/understanding-kubernetes-controllers-part-i-. . .

27 **Understanding Kubernetes controllers, part II** LeftAsExercise, July 2019: object stores and indexers
leftasexercise.com/2019/07/11/understanding-kubernetes-controllers-part-ii-. . .

28 **Understanding Kubernetes controllers, part III** LeftAsExercise, July 2019: informers
leftasexercise.com/2019/07/15/understanding-kubernetes-controllers-part-iii-informers/

29 **LeftAsExercise: all Kubernetes posts** Blog category: the full series
https://leftasexercise.com/category/kubernetes/

30 **kubernetes/sample-controller** controller.go: a controller built on client-go directly
github.com/kubernetes/sample-controller/.../controller.go#L328-L338

$ Sources (7/8)

31 **How LinkedIn moved its Kubernetes APIs to a different API group** LinkedIn Engineering blog
linkedin.com/blog/engineering/infrastructure/how-linkedin-moved-its-kubernetes-apis-to-. . .

32 **Software Engineering at Google, ch. 1: Time and Change** Titus Winters et al., abseil.io (O'Reilly, 2020)
https://abseil.io/resources/swe-book/html/ch01.html#time_and_change

33 **Versions in CustomResourceDefinitions** Kubernetes documentation
kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definition-versioning/

34 **Groups and Versions and Kinds, oh my!** The Kubebuilder Book, CronJob tutorial (gvks.md)
github.com/kubernetes-sigs/kubebuilder/.../cronjob-tutorial/gvks.md

$ Sources (8/8)

35 **Multi-version API: conversion concepts** The Kubebuilder Book
https://book.kubebuilder.io/multiversion-tutorial/conversion-concepts

36 **kubernetes-sigs/cluster-api** Cluster API: declarative cluster lifecycle, a large multi-version operator
https://github.com/kubernetes-sigs/cluster-api

37 **sigs.k8s.io/cluster-api/util/conversion** Cluster API: MarshalData / UnmarshalData, the conversion-data annotation
https://pkg.go.dev/sigs.k8s.io/cluster-api/util/conversion

38 **memgraph/kubernetes-operator** Source code, docs and comments in the repo
https://github.com/memgraph/kubernetes-operator